The difference between the two alternatives is that the first one simply unmarks a key node, making it an intermediate node, and doesn't worry about the tree structure: this can be easily implemented reusing the `searchNode` method in listing 6.4, as we show in listing 6.7.

What's the issue with this method? Take a look at figure 6.8: if we just transform a key node `N` into an intermediate node, we can have two possible situations:

- `N` is an inner node, so it will have children storing keys that can only be reached passing through `N` itself;
- `N` is a leaf: this means that there is no key stored in the trie that can only be reached through `N`.

The latter case is illustrated in figure 6.9: after "unmarking" the key node at the end of the path, we can see that the trie has a "dangling" branch that contains no key. While it is perfectly fine to leave such branch, since all the methods for manipulating tries will still work, if data is dynamic and there is a large ratio of deletions on the trie, the amount of memory wasted in dangling branches can become significant.